CHAPTER 1
INTRODUCTION

In today's digital era, managing and verifying student records is a critical task for educational institutions. Traditional methods of storing student data are often centralized and vulnerable to data tampering, unauthorized access, and lack of transparency. These limitations can lead to issues such as data manipulation, delays in verification, and reduced trust in academic records.

To overcome these challenges, this project introduces a Blockchain Student Record System, which utilizes blockchain-inspired concepts along with modern web technologies. Blockchain technology is known for its decentralized, secure, and immutable nature, making it an ideal solution for maintaining sensitive data such as student records.

The proposed system is developed using the MERN stack, which includes MongoDB, Express.js, React, and Node.js. It incorporates SHA-256 encryption to ensure data security and integrity. Once a student submits their profile, the information is reviewed by a teacher. After successful verification, the data is encrypted and stored in a blockchain-like structure, where it becomes immutable and cannot be altered.

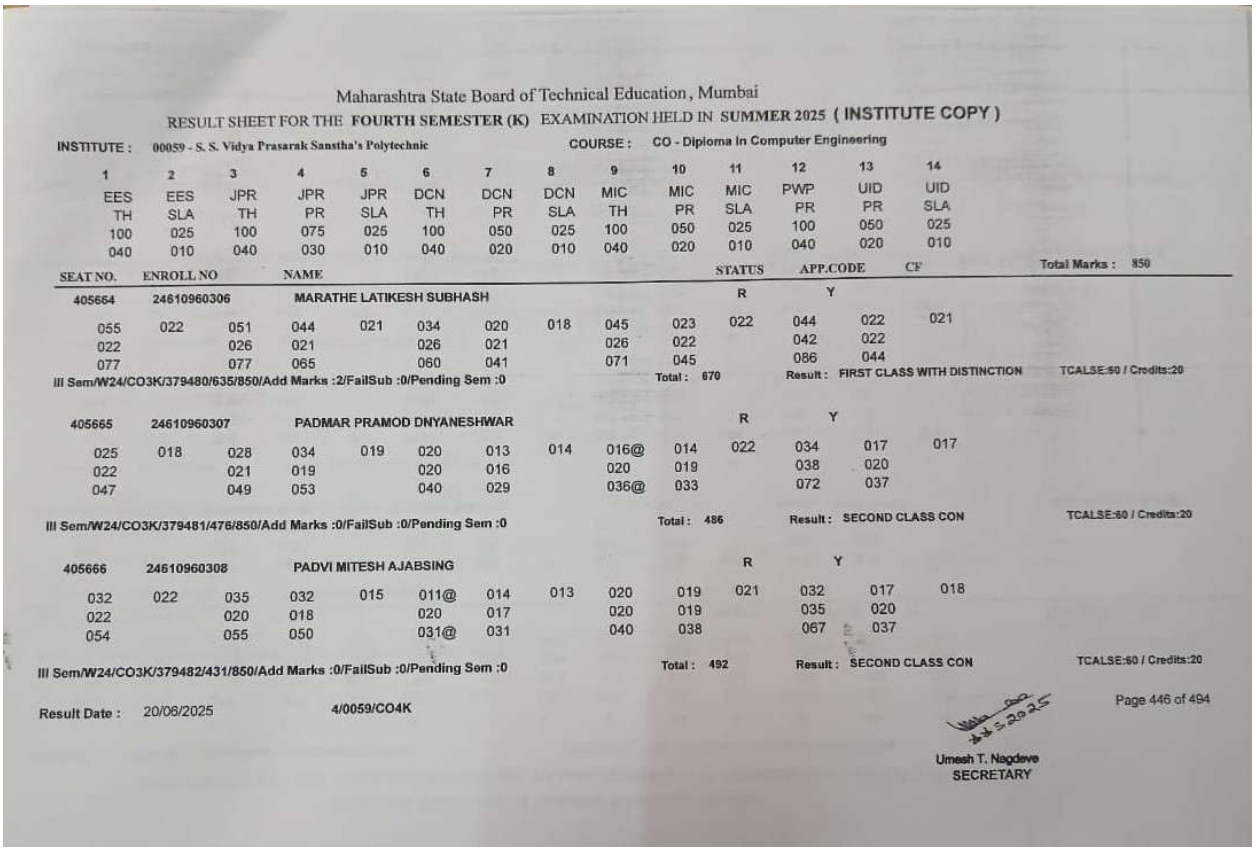
The system also provides role-based access control, allowing students, teachers, and administrators to interact with the system according to their responsibilities. Features such as real-time status tracking, change request management, and secure authentication using JSON Web Tokens (JWT) enhance the overall functionality and user experience.

This project aims to provide a secure, transparent, and efficient solution for managing student records, ensuring data integrity and building trust within educational systems.

CHAPTER 2
LITERATURE SURVEY

2.1 Study of Existing System

We studied the existing system used for managing student records in educational institutions. In many colleges and universities, student data such as personal details, academic records, and verification status are still maintained using traditional database systems or even manual paper-based methods.



Maharashtra State Board of Technical Education, Mumbai

RESULT SHEET FOR THE FOURTH SEMESTER (K) EXAMINATION HELD IN SUMMER 2025 (INSTITUTE COPY)

INSTITUTE : 00059 - S. S. Vidya Prasarak Sanstha's Polytechnic COURSE : CO - Diploma In Computer Engineering

1	2	3	4	5	6	7	8	9	10	11	12	13	14
EES	EES	JPR	JPR	JPR	DCN	DCN	DCN	MIC	MIC	MIC	PWP	UID	UID
TH	SLA	TH	PR	SLA	TH	PR	SLA	TH	PR	SLA	PR	PR	SLA
100	025	100	075	025	100	050	025	100	050	025	100	050	025
040	010	040	030	010	040	020	010	040	020	010	040	020	010

Total Marks : 850

SEAT NO.	ENROLL NO	NAME	STATUS	APP.CODE	CF
405664	24610960306	MARATHE LATIKESH SUBHASH	R	Y	
055 022 051 044 021 034 020 018 045 023 022 044 022 021					
022 026 021 026 021 026 022 042 022					
077 077 065 060 041 071 045 086 044					
III Sem/W24/CO3K/379480/635/850/Add Marks :2/FailSub :0/Pending Sem :0			Total : 670	Result : FIRST CLASS WITH DISTINCTION	TCALSE:60 / Credits:20
405665	24610960307	PADMAR PRAMOD DNYANESHWAR	R	Y	
025 018 028 034 019 020 013 014 016@ 014 022 034 017 017					
022 021 019 020 016 020 019 038 020					
047 049 053 040 029 036@ 033 072 037					
III Sem/W24/CO3K/379481/476/850/Add Marks :0/FailSub :0/Pending Sem :0			Total : 486	Result : SECOND CLASS CON	TCALSE:60 / Credits:20
405666	24610960308	PADVI MITESH AJABSING	R	Y	
032 022 035 032 015 011@ 014 013 020 019 021 032 017 018					
022 020 018 020 017 020 019 035 020					
054 055 050 031@ 031 040 038 067 037					
III Sem/W24/CO3K/379482/431/850/Add Marks :0/FailSub :0/Pending Sem :0			Total : 492	Result : SECOND CLASS CON	TCALSE:60 / Credits:20

Result Date : 20/06/2025 4/0059/CO4K

Page 446 of 494

Umesh T. Nagdeve
SECRETARY

2.1 Student Result Sheet

This is an official result sheet from MSBTE for 4th semester Diploma in Computer Engineering (Summer 2025). It shows students' subject-wise marks (theory, practical, internal), total out of 850, final result (e.g., distinction/second class), along with enrollment, verification, and credit details.

Class : First Year

Branch : Civil Engineering

Shift : First Shift (0059)

Sr. No.	Student ID	Enrollment no	Name of student	Gender	Date Of Admission	Name Of HighSchool/College	Previous Exam Marks S.S.C	Date Of Birth	Permenent Address	Contact No	Religion	Cast	Category
23	P00CE2023-8628	23610960023	MALI HARSHAL ANAND	MALE	03/08/2023	SHRI T.T.K MAHAJAN HIGH SCHOOL	342	26/12/2007	PRIYDARSHANI NAGAR NAGAONBARI DEOPUR DHULE	9284380066	HINDU	MALI	OPEN
24	P00CE2023-8566	23610960024	MALI NIKITA DIPAK	FEMALE	29/07/2023	KAMALABAI GIRLS HIGH SCHOOL,DHULE	293	01/05/2006	VIDUYT NAGAR PLOT NO 13 STATION ROAD DHULE	9307272352	HINDU	HINDU MALI	OBC
25	P00CE2023-8684	23610960025	MANGIDKAR VEENA MANOJ	FEMALE	12/08/2023	CANOSSA CONVENT SCHOOL SOCIETY,DHULE	303	08/08/2007	51,MARUTI NAGAR CHALISGAON ROAD,DHULE	8605008605	HINDU	YELMAR	OPEN
26	P00CE2023-8776	23610960026	MARATHE HARSHVARDHAN KAILAS	MALE	25/08/2023	JA. HIND HIGH SCHOOL DHULE	333	18/02/2008	LANE NO 7 HOOJSE NO 2146B NEAR MAR MATA MANDIR DHULE	7020525431	HINDU	MARATHA	OPEN
27	P00CE2023-8845	23610960027	MISTARI KALPESH PRABHAKAR	MALE	29/08/2023	K.S.K NEW CITY HIGH SCHOOL	308	03/04/2006	PLOT NO 30 RAMNAGAR NEAR ABHAY COLLEGE MARKET YARD DHULE	8766773024	HINDU	SUTAR	OBC
28	P00CE2023-8772	23610960028	ANSARI MOHAMMAD SAAD JAVEED HUSAIN	MALE	25/08/2023	ANGLO URDU HIGH SCHOOL	325	06/01/2007	LANE NO 14 MADHAVPURA NEAR DARUL KAZA DHULE	9773019774	MUSLIM	JULAHA	OBC
29	P00CE2023-8624	23610960029	MORE ADITYA RATAN	MALE	02/08/2023	SARVODAYA MADHYAMIK VIDYALA, KAVPIMPRI	420	02/09/2007	INDRAPIMPRI POST KAVPIMPRI AMALNER	9561852179	HINDU	CHAMBHAR	SC
30	P00CE2023-8797	23610960030	MORE KUNAL CHANDRASHEKHAR	MALE	25/08/2023	JAI HIND HIGH SCHOOL DHULE	372	23/09/2007	LANE NO C1 HOUSE NO 178/A VANJAR GALLI PAT BAJAR SUBHASH NAGAR OLD DHULE	8055324903	HINDU	KUNBI	OBC
31	P00CE2023-8774	23610960031	MORE VASUDHA VASANT	FEMALE	25/08/2023	CANOSSA CONVENT HIGH SCHOOL, CHALISGAON ROAD, DHULE.	340	15/02/2007	PLOT NO. 9, ASHOK NAGAR, CHITTOD ROAD DHULE.	7888260151	HINDU	MAHAR	SC
32	P00CE2023-8738	23610960032	SAYYAD NAZNEEN AKHTAR ASIF	FEMALE	16/08/2023	NOOR URDU HIGH SCHOOL & JR. COLLEGE OF ARTS, DHULE.	265	19/06/1996	NASHEMAN PLOT NO. 21, ALHERA COLONY OPPOSITE ALHERA URDU SCHOOL DHULE	8669077660	MUSLIM	MANIYAR	OPEN
33	P00CE2023-8723	23610960033	PARDESHI BHAVESH RAJENDRA SING	MALE	14/08/2023	ST. ANNS ENGLISH SCHOOL, DHULE.	369	27/08/2007	CHALISGAON ROAD, MAROTI NAGAR, DHULE.	8080271753	HINDU	HINDU PARDESHI	OBC
34	P00CE2023-8685	23610960034	PATEL PANKAJ YOGESH	MALE	12/08/2023	RAJE CHHATRAPATI SAMBHAJI VIDYALAYA DHULE.	303	28/08/2007	AT POST VARKHEDE, TAL DIST-DHULE.	7709324273	HINDU	HINDU GUJAR	OBC
35	P00CE2023-8622	23610960035	PATIL DNYANESHWAR ANIL	MALE	02/08/2023	SARVODAY MADHYAMIK VIDYALAYA, KAVPIMPRI	384	07/12/2007	AT INDRAPIMPRI POST KAVPIMPRI TA. AMALNER DIST JALGAON	8551843775	HINDU	KUNBI	OBC
36	P00CE2023-8786	23610960036	PATIL KALPESH ASHOK	MALE	25/08/2023	Z.D. SONAWANE HIGH SCHOOL MUDI, TA.-AMALNER, DIST.-JALGAON.	369	21/08/2006	VTC-BORADE, PO-MUDI, AMALNER, JALGAON.	7822948182	HINDU	KUNBI	OBC

Page 3 of 5

2.2 Personal Details sheet of Student

This document is a student record sheet for First Year Civil Engineering students, containing personal, academic, and demographic details such as enrollment, admission date, SSC marks, and contact information. It also includes attributes like gender, religion, caste, and category, making it useful for student management and verification.

We also interacted with students, teachers, and admin staff to understand current data management and verification processes.

Q1. How are student records maintained in institutions?

Q2. Is the data secure and protected from unauthorized access?

Q3. How do teachers verify student information?

Q4. What problems occur in the current system?

Q5. Is there any use of blockchain or advanced security methods?

Q6. How is student data updated or modified?

Q7. Can students track their verification status?

After discussing with them, we learnt that:

Ans 1: Student records are mostly maintained in centralized databases or sometimes manually using paper documents.

Ans 2: Data security is limited and can be vulnerable to unauthorized access or tampering.

Ans 3: Teachers verify student information manually by checking documents and records.

Ans 4: Problems include data manipulation, lack of transparency, and difficulty in tracking records.

Ans 5: Most institutions do not use blockchain or advanced encryption techniques.

Ans 6: Data can be modified easily, sometimes without proper tracking or audit.

Ans 7: Students usually cannot track real-time verification status.

The existing system is simple but has several limitations such as lack of security, transparency, and proper tracking mechanisms.

2.2 Limitations of Existing System

The current student record system has several issues affecting its efficiency. It lacks proper data security, making records vulnerable to unauthorized access and tampering. There is no immutability, so data can be easily changed, reducing trust. The system also lacks transparency, as students cannot track verification status.

Handling large amounts of data is difficult and time-consuming. Manual verification increases workload and delays the process. Human errors during data entry can affect accuracy. Additionally, there is no proper system to track changes made to records.

Summary of Limitations

• Lack of data security and protection

• No immutability of records

• Lack of transparency in verification process
• Difficult to manage large data
• Time-consuming manual verification
• High chances of human errors
• No proper audit trail or tracking system

2.3 Problem Identification

The study of the existing system shows that there are several problems faced by students, teachers, and administrators.

Problems of Students

• Students cannot track verification status in real-time
• Difficulty in updating or correcting data
• Lack of transparency in the system
• No proper system for submitting change requests

Problems of Teachers

• Difficult to verify large numbers of student records
• Manual verification increases workload
• No proper system to track changes or history
• Time-consuming process

Problems of Admin/Staff

• Difficulty in managing and organizing large data
• Manual processes are inefficient
• No centralized secure system
• Difficulty in monitoring system activities

Problems of Existing System

• Data can be modified or tampered
• No blockchain or encryption-based security
• Lack of audit trail

• No real-time tracking

• Inefficient data management

Overall Problems

• Lack of secure and tamper-proof system

• Inefficient verification process

• Poor data transparency

• Time-consuming operations

• Risk of data manipulation

2.4 Problem Definition

The existing system for managing student records is not secure, transparent, or efficient. It lacks proper data protection, tracking, and verification mechanisms.

The main objective of this project is to develop a Blockchain for Student record System that ensures secure, transparent, and immutable storage of student records.

The proposed system aims to solve the problems of the existing system by providing a blockchain-inspired solution with encryption and role-based access.

The system will focus on:

a) Secure storage of student data using encryption

b) Immutable record management using blockchain concepts

c) Efficient verification process by teachers

d) Real-time tracking of verification status

To achieve this, the system will provide:

a) Encrypted data storage using SHA-256

b) Blockchain-like structure for storing records

c) Role-based access for students, teachers, and admin

d) Change request mechanism for data updates

e) Centralized and secure database system

Benefits of Proposed System

- Ensures data security and integrity
- Prevents unauthorized data modification
- Improves transparency and trust
- Reduces manual work

CHAPTER 3
SCOPE OF THE PROJECT

3.1 Scope of the Project

The proposed system, Blockchain Student Verification System, is designed to simplify and improve the process of managing and verifying student records in educational institutions. In the existing system, student data is often managed manually or through traditional centralized databases, which leads to problems such as data tampering, lack of transparency, and inefficient verification processes. The scope of this project focuses on overcoming these limitations by providing a secure, transparent, and efficient digital solution.

The application provides a centralized platform where all student-related information can be stored, managed, and verified securely using blockchain-inspired technology. This system helps students, teachers, and administrators to maintain accurate and tamper-proof academic records.

The scope of the project includes managing all aspects of student data, starting from registration to profile verification and maintaining a complete record history. The system ensures that all data is encrypted, securely stored, and accessible whenever required.

Key Functional Areas of the System

The scope of this project is divided into several functional modules which help in efficient management of the system.

1) Student Registration and Management

• The system allows students to register with basic details such as name, enrollment number, and contact information.

• Each student has a unique record in the system.

• It helps in maintaining organized and structured student data.

• Students can update and view their profile before verification.

2) Student Profile Management

• The system provides a multi-section profile form (basic, contact, academic details, etc.).

• Students can enter complete academic and personal information.

• Profile data is validated before submission.

• Once submitted, the profile is locked after verification.

3) Blockchain-based Record Storage

• Verified student data is encrypted using SHA-256.

• Data is stored in a blockchain-like structure as immutable blocks.

• Each block is linked with the previous block to ensure data integrity.

• Once stored, the data cannot be modified, ensuring tamper-proof records.

4) Verification System (Teacher Module)

• Teachers can review student profiles section by section.

• Teachers can approve or reject student profiles.

• Feedback can be provided for corrections.

• It ensures proper validation of student data.

5) Change Request Management

• Students can request changes before final verification.

• Teachers can approve or reject change requests.

• All changes are tracked for transparency.

• It maintains a proper audit trail of modifications.

6) Authentication and Security

• Secure login system using JWT authentication.

• Role-based access for students, teachers, and admin.

• Encrypted data ensures privacy and protection.

• Prevents unauthorized access to sensitive data.

7) Dashboard and Analytics

• Real-time tracking of verification status.

• Dashboard shows statistics like verified and pending students.

• Helps administrators monitor system performance.

• Improves decision-making and management.

Benefits of the Proposed System

• Ensures data security and integrity

• Prevents unauthorized modification of records

• Reduces manual work and paperwork

• Improves transparency in verification process

• Saves time for teachers and administrators

• Provides real-time tracking of student status

• Enhances overall educational data management

Future Scope of the Project

The system can be further enhanced with additional features such as:

• Integration with real blockchain networks (e.g., Ethereum)

• Mobile application for easy access

• SMS and email notification system

• Advanced analytics and reporting tools

• Two-factor authentication (2FA) for enhanced security

• Integration with university or government education systems

• Cloud-based deployment for scalability and accessibility

CHAPTER 4
METHODOLOGY

4.1 Requirements Gathering and Analysis

The Software Development Life Cycle (SDLC) followed in this project includes the following phases:

1. Requirements Gathering and Analysis
2. System Design
3. Implementation
4. Testing
5. Deployment
6. Maintenance

Phase 1: Requirements Gathering and Analysis

In this phase, we focused on understanding the needs of students, teachers, and administrators. The main objective was to identify the problems in the existing system and define the requirements of the proposed system.

We collected information through:

• Discussions with students

• Interaction with teachers

• Observation of existing record management systems

This helped us to design a system that solves real-world problems effectively.

4.1.1 User (Student) Requirements

The system should fulfill the following requirements for students:

• The system should allow students to register and login securely.

• It should store information such as personal and academic details.

• Students should be able to fill and submit their profile.

• The system should allow students to track verification status.

• Students should be able to request changes in their profile.

• The system should be simple and user-friendly.

4.1.2 Teacher Requirements

• The system should allow teachers to review student profiles.

• Teachers should be able to verify or reject student data.

• The system should allow providing feedback for corrections.

• Teachers should be able to track verification status easily.

• It should reduce manual workload and improve efficiency.

4.1.3 Verification Requirements

• The system should provide a proper verification workflow.

• It should allow teachers to approve or reject profiles.

• It should track verified and pending records.

• It should maintain a history of verification actions.

• It should ensure that verified data becomes immutable.

4.1.4 Record Management Requirements

• The system should store all data in a structured format.

• It should allow adding, updating (before verification), and viewing records.

• It should maintain complete student profile history.

• It should provide easy and fast access to data.

4.1.5 System Requirements

Software Requirements

Operating System: Windows OS

Platform: Visual Studio Code

Languages: HTML, CSS, JavaScript, Node.js

Frontend: React

Database: MongoDB

Hardware Requirements

Processor: Intel i3 or higher

RAM: Minimum 4 GB

Hard Disk: 500 GB or more

Phase 2: System Design

In this phase, the system architecture, database structure, and user interface were designed.

• Database design includes collections for students, profiles, and verification data

• User interface is designed for easy navigation and usability

• System modules are clearly defined (Student, Teacher, Admin)

Phase 3: Implementation

In this phase, the system is developed using:

• Frontend: React, HTML, CSS, JavaScript

• Backend: Node.js, Express.js

• Database: MongoDB

All modules such as registration, profile management, verification, and blockchain storage are implemented.

Phase 4: Testing

Testing is done to ensure the system works correctly.

• Unit Testing – testing individual modules

• Integration Testing – testing combined modules

• System Testing – testing the complete system

Errors and bugs are identified and fixed.

Phase 5: Deployment

• The system is deployed on a server

• Users can access the system through a web browser

• The system becomes ready for real-time use

Phase 6: Maintenance

• Regular updates are performed

• Bugs are fixed if found

• System performance is improved over time

• Security updates are applied when needed

CHAPTER 5

DETAILS OF DESIGN, WORKING AND PROCESSES

System Design:

In the System Design phase of the Blockchain Student Verification System, the main objective was to convert the gathered requirements into a structured and well-organized system. This phase plays a crucial role in defining how the system will function and how different components will interact with each other.

The system design includes:

• System Architecture

• Database Design

• Module Design

• User Interface Design

The design ensures that the system is secure, efficient, user-friendly, and capable of handling all required operations smoothly.

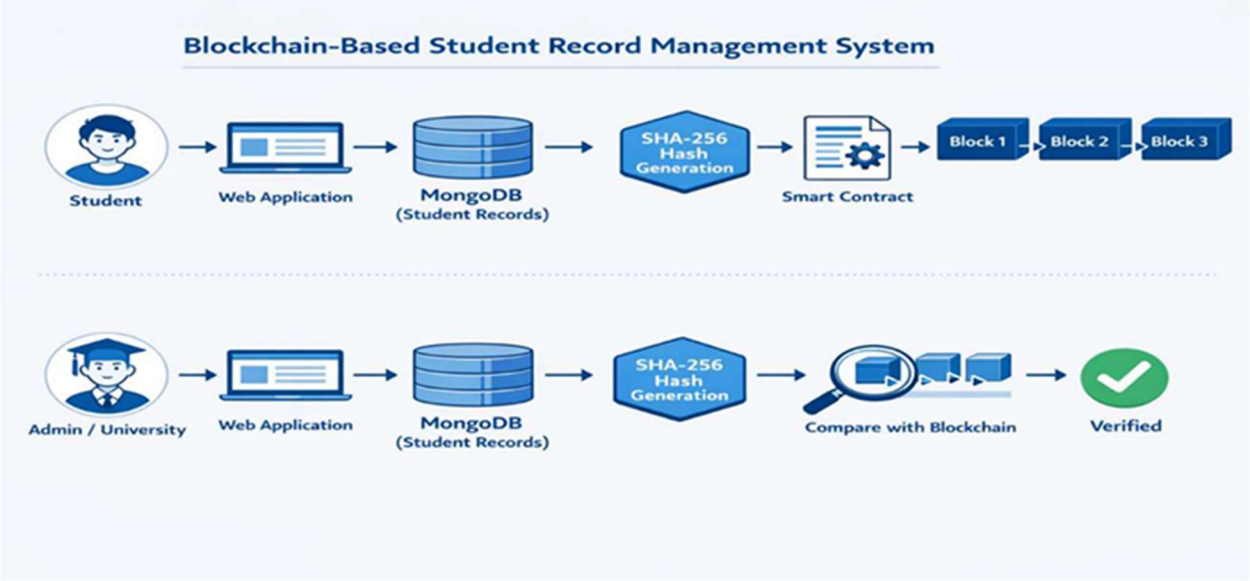


Fig 5.1: System Workflow Diagram

The above figure represents the overall working of the Blockchain Student Verification System, which involves three main users: Student, Teacher, and Admin. The system is designed to ensure secure data handling and efficient verification of student records.

Initially, the process starts with the student. The student registers into the system, logs in, and fills their profile with personal and academic details. After submitting the profile, the data is stored temporarily and sent for verification.

Once submitted, the teacher reviews the student profile. The teacher can either approve (verify) or reject the profile with feedback. If approved, the system encrypts the data using SHA-256 and stores it in a blockchain-like structure, making it immutable.

After verification, the student can view their verified profile and status. If rejected, the student can update the details based on feedback and resubmit the profile.

The admin plays a vital role in managing the entire system. The admin manages students, teachers, and system operations, and monitors overall activities.

Step-wise Working of System

1. Student registers and logs into the system
2. Student fills profile details
3. Student submits profile for verification
4. Data is stored temporarily in database
5. Teacher reviews the student profile
6. Teacher approves or rejects the profile
7. If approved → Data is encrypted (SHA-256)
8. Encrypted data is stored as blockchain block
9. Profile becomes immutable (locked)
10. Student views verification status
11. If rejected → Student updates and resubmits
12. Admin manages users and monitors system

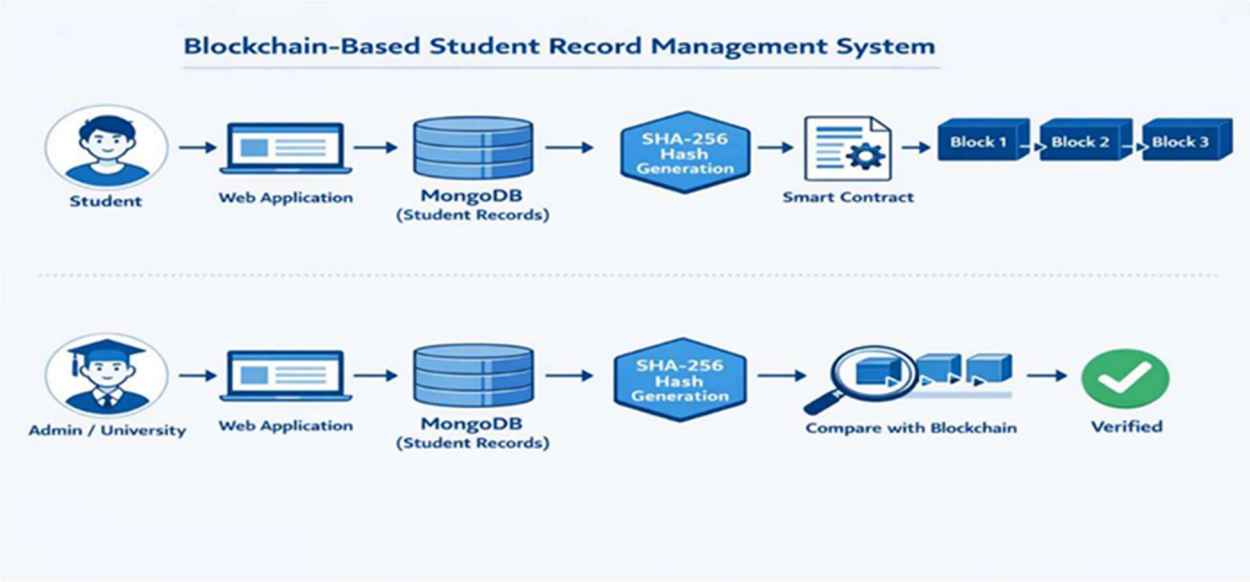


Fig 5.2: System Architecture Diagram

The System Architecture Diagram represents the overall structure of the Blockchain Student Verification System. It shows how different users interact with the system through their respective dashboards.

The system starts with a landing page, where users log in based on their roles. The system is divided into three main modules: Student Dashboard, Teacher Dashboard, and Admin Dashboard. Each dashboard provides specific functionalities according to user roles.

The architecture ensures secure data flow, role-based access, and efficient communication between system components.

Modules in System Architecture

1. Student Dashboard

- Register and login
- Fill and manage profile
- Submit profile for verification
- View verification status
- Send change requests

2. Teacher Dashboard

- View student profiles
- Verify or reject student data

• Provide feedback

• Manage verification requests

• View verified students

3. Admin Dashboard

• Manage students and teachers

• Monitor system activities

• View reports and analytics

• Handle password reset requests

Key Features of Architecture

• Role-based access control system

• Blockchain-based secure storage

• Encrypted data using SHA-256

• Centralized database management

• Real-time verification tracking

• Easy navigation through dashboards

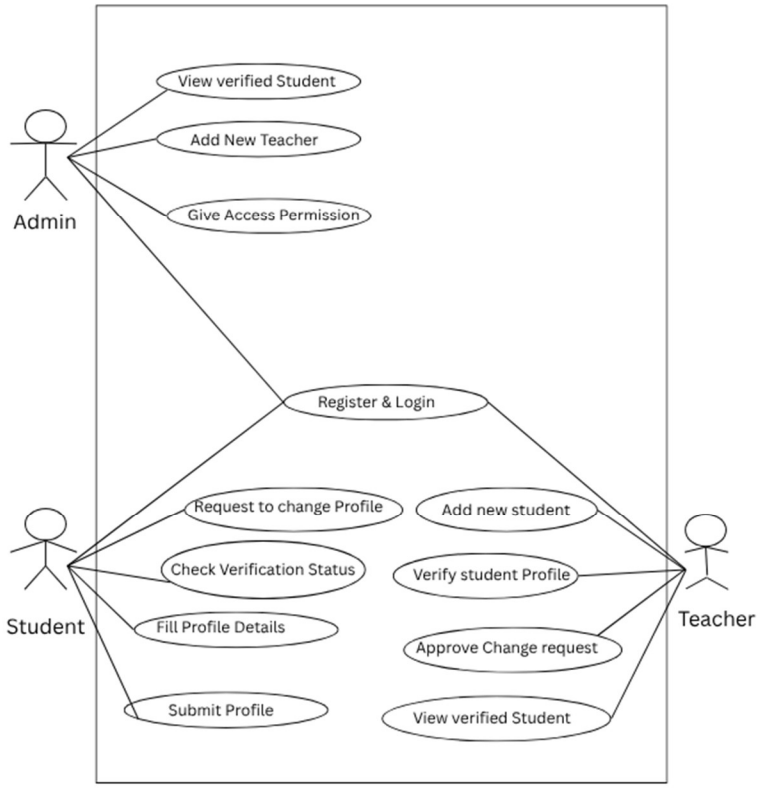


Fig 5.3: Use Case Diagram

Actors:

- Student: Represents the user who manages their profile and submits data
- Teacher: Responsible for verifying student records
- Admin: Manages the entire system

Use Cases:

Authentication

- Register: Student creates account
- Login: User logs into system
- Verification: System verifies credentials

Student Activities

- Fill Profile: Enter personal and academic details
- Submit Profile: Send data for verification
- View Status: Check verification result
- Request Changes: Submit modification request

Teacher Activities

- View Student Data: Access student profiles
- Verify Profile: Approve student data
- Reject Profile: Send feedback for correction
- Manage Requests: Handle change requests

Admin Activities

- Manage Students: Control student data
- Manage Teachers: Add/remove teachers
- Monitor System: Track system activity
- Generate Reports: Analyze system data

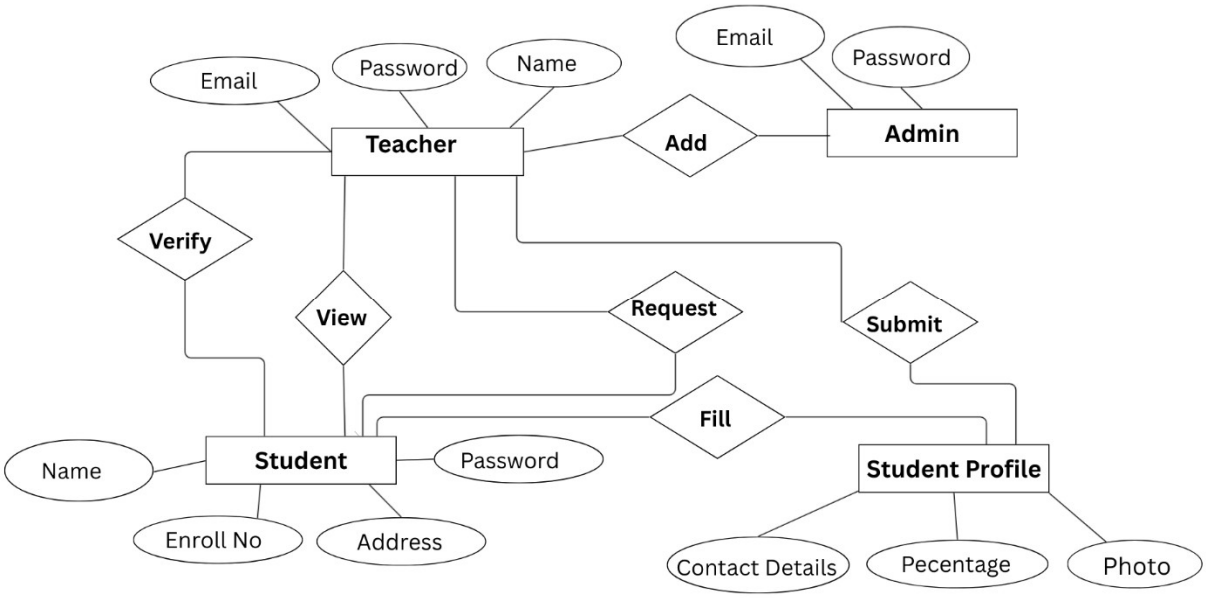


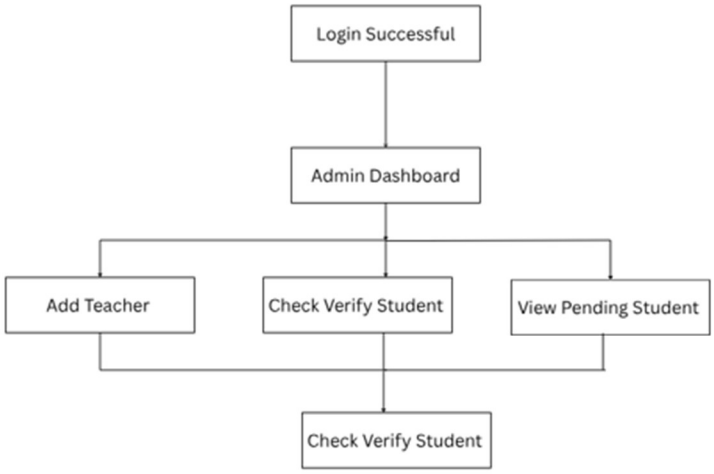
Fig 5.4: ER Diagram

The ER diagram represents the structure of the Blockchain for Student record System and shows the relationships between different entities such as Student, Teacher, Admin, Student Profile, and Blockchain Blocks.

Each student has one profile, and each verified profile is stored as a block in the blockchain structure. The block contains encrypted data and is linked with the previous block using a hash reference.

The system ensures proper data organization, secure storage, and efficient management of student records while maintaining data integrity and immutability.

5.5 Flow Diagram of the Admin Module:



5.5 Flow Diagram of the Admin Module:

This diagram represents the data flow of the Admin Module in the student verification system. It begins with successful admin login, after which the admin gains access to the dashboard that serves as the central control unit. From the dashboard, the admin can perform key operations such as adding teachers, checking verified student records, and viewing pending student requests. The system allows continuous monitoring of student verification status, ensuring proper management and control over academic records and verification processes.

Step-wise Explanation

1. **Login Successful**

Admin enters valid credentials and logs into the system

2. **Admin Dashboard Access**

Admin is redirected to the dashboard

Dashboard acts as the main control panel

3. **Add Teacher**

Admin adds new teachers to the system

Teachers help in verifying student records

4. **Check Verified Students**

Admin views students who are already verified

Helps in tracking completed records

5. **View Pending Students**

Admin checks students whose verification is still pending

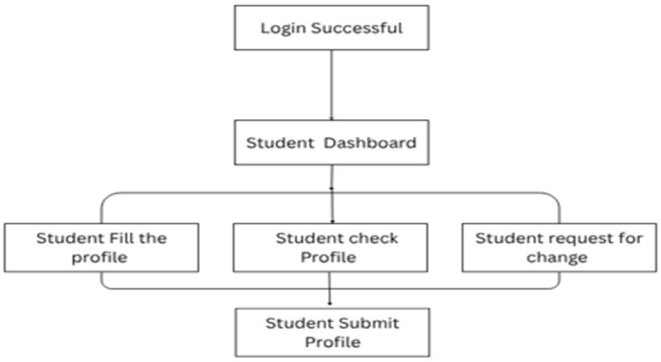
These require further action

6. **Verification Monitoring**

Admin continuously monitors student verification status

Ensures data accuracy and proper workflow

5.6 Flow Diagram of the Student Module:



5.6 Flow Diagram of the Student Module:

This diagram represents the data flow of the Student Module in the system. It starts with successful student login, after which the student accesses the dashboard. From the dashboard, the student can perform actions such as filling their profile, checking their profile details, and requesting changes if needed. Once all details are completed and verified by the student, the profile is submitted to the system for further processing and verification. This flow ensures that student data is properly entered, reviewed, and finalized before submission.

Step-wise Explanation:-

1. **Login Successful**

Student logs into the system using valid credentials

2. **Student Dashboard Access**

Student is redirected to the dashboard

Dashboard acts as the main interface

3. **Fill Profile**

Student enters personal, academic, and contact details

4. **Check Profile**

Student reviews the entered information

Ensures all data is correct

5. **Request for Change**

Student can request modifications if needed

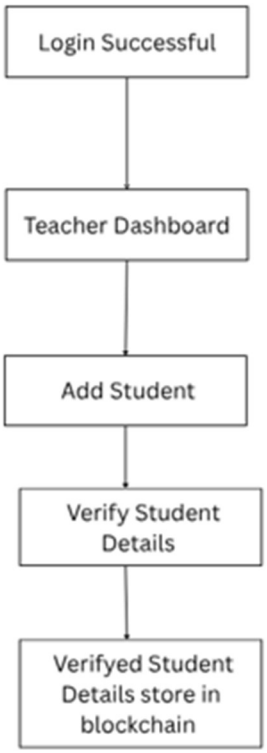
Useful before final submission

6. **Submit Profile**

Student submits the final profile

Data is sent for verification

5.7 Flow Diagram of the Teacher Module:



5.7 Flow Diagram of the Teacher Module:

This diagram represents the data flow of the Teacher Module in the student verification system. It begins with successful teacher login, after which the teacher accesses the dashboard. From the dashboard, the teacher can add student records into the system and verify student details. Once the verification process is completed, the verified student data is securely stored in the blockchain. This ensures that the data becomes immutable and tamper-proof, maintaining integrity and authenticity of student records.

Step-wise Explanation

1. **Login Successful**

- Teacher logs into the system using valid credentials

2. **Teacher Dashboard Access**

- Teacher is redirected to the dashboard

3. Dashboard acts as the control panel

- **Add Student**

4. Teacher adds student basic details into the system
5. Initial record is created

- **Verify Student Details**

6. Teacher checks and validates student information
7. Ensures accuracy of data

- **Store in Blockchain**

8. After verification, data is securely stored
9. Blockchain ensures data is immutable and tamper-proof

CHAPTER 6
RESULTS AND APPLICATIONS

The **Blockchain Student Record System** has been successfully developed and implemented as a web-based application to manage and verify student records efficiently. The system provides a secure digital platform for students, teachers, and administrators to interact and manage student-related data in a structured and organized manner.

The application is developed using HTML, CSS, JavaScript, React, and Node.js, with a backend database managed using MongoDB. The system ensures secure data handling through SHA-256 encryption, easy accessibility, and efficient verification of student records.

1. Student Module Results

The student module allows users to manage their profile and verification process effectively.

• Students can register and login into the system

• They can fill and manage their profile details

• Submit profile for verification

• View verification status (approved/rejected)

• Request changes if required

This module helps students to maintain accurate records and track their verification status easily.

2. Teacher Module Results

The teacher module improves the verification process and ensures data accuracy.

• Teachers can view student profiles

• Review student details section-wise

• Approve or reject student profiles

• Provide feedback for corrections

• Monitor verification progress

This reduces manual work and ensures proper validation of student data.

3. Admin Module Results

The admin module provides complete control over the system.

• Manage students and teachers

• Monitor system activities

• Maintain database records

• Handle verification and user-related issues

• Generate reports and analytics

The admin ensures smooth functioning, security, and overall management of the system.

Applications of the System

• Educational institutions for secure student record management

• Universities for verification of academic data

• Online certificate validation systems

• Government or private educational databases

• Digital identity verification platforms

The system enhances data security, transparency, and efficiency, making it suitable for modern educational environments.

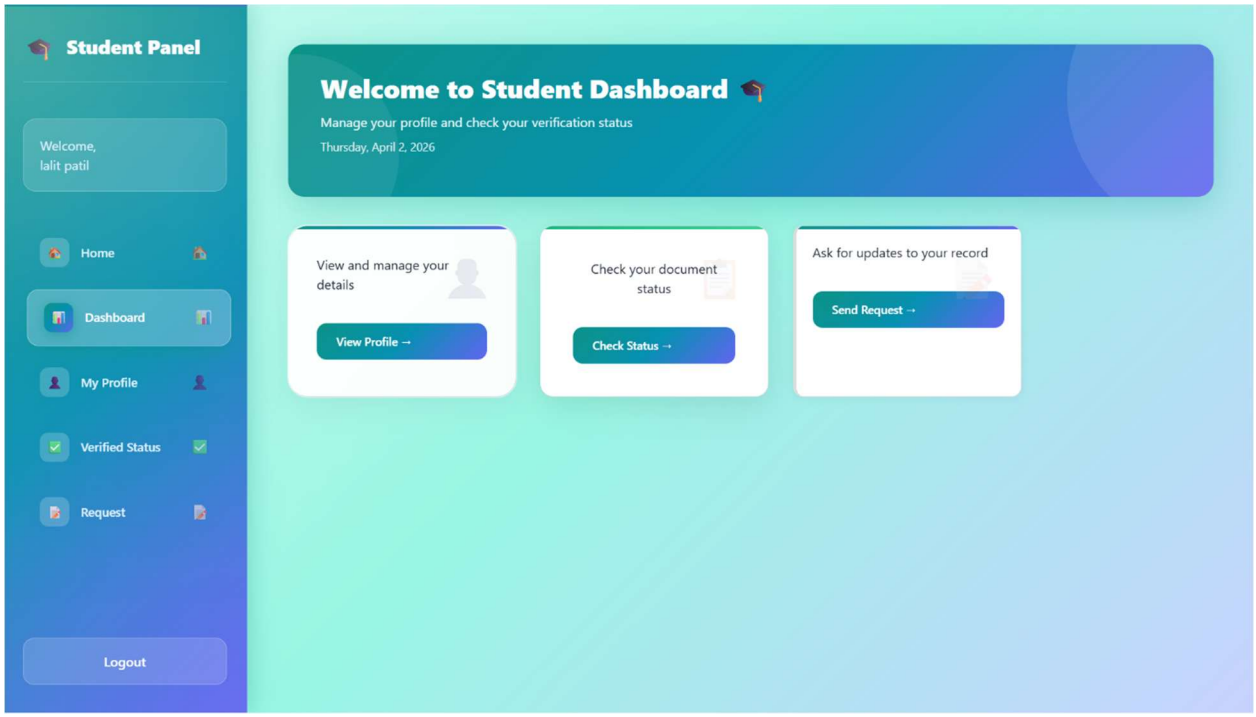


Fig 6.1: Student Dashboard Interface

- Represents the Student Dashboard of the Blockchain Student Verification System
- Provides a user-friendly interface for managing profile and verification tasks
- Displays important details like profile status (verified/pending) and recent updates
- Left-side navigation menu includes:

1. Dashboard
2. Profile
3. Verification Status
4. Change Request
5. Settings

- Main section shows profile summary using cards
- Helps students quickly understand approval, pending, or required changes status
- Displays notifications and teacher feedback
- Enables students to track verification progress easily
- Ensures a smooth and transparent verification process

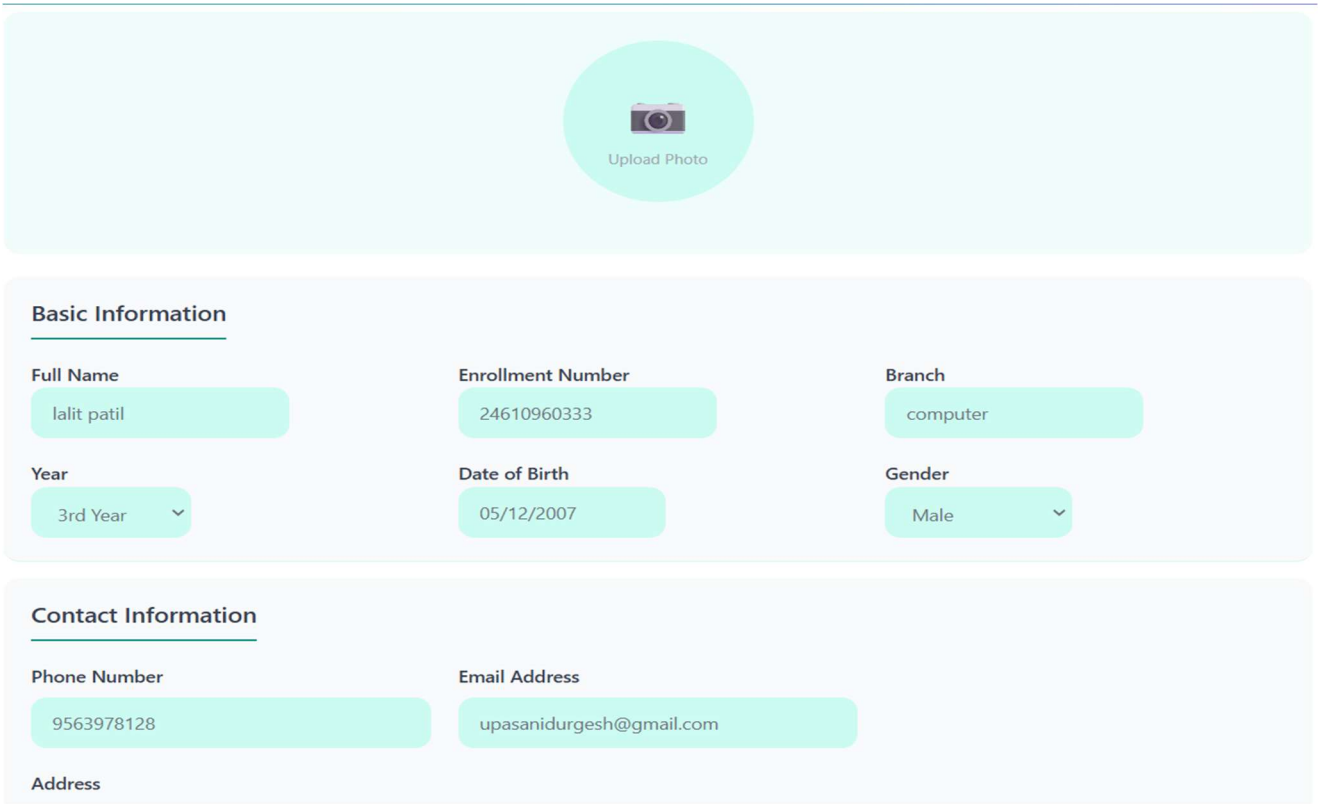


Fig 6.2: Student Profile Interface

- Represents the Student Profile Interface of the Blockchain Student Verification System
- Allows students to enter and manage personal & academic details
- Includes fields like name, enrollment number, contact details, academic records
- Displays basic information for confirmation
- Organizes data into sections:

1. Basic Details
2. Contact Information
3. Guardian Details
4. Academic Information

- Designed to be simple and user-friendly
- Reduces chances of data entry errors
- Supports validation to ensure all required fields are filled
- Helps in efficient profile creation and management
- Ensures smooth and reliable verification process

Code: Fetching Profile

```
const fetchProfile = async () => {
        setIsLoading(true);
        try {
          const token = localStorage.getItem("token");
          const user = JSON.parse(localStorage.getItem("user")
    || "{}");
          const enroll = user?.enroll;
          const studentRes = await fetch(
            `http://localhost:8000/api/student/enroll/${enroll}
    `
          );
          const studentData = await studentRes.json();

          console.log("Enroll API:", studentData);
```

```
      if (studentRes.ok && (studentData.data ||
studentData.student)) {
  const s = studentData.data || studentData.student;
        setForm((prev) => ({
          ...prev,
          name: s.name || "",
          enroll: s.enroll || "",
          branch: s.branch || "",
          year: s.year || "",
          dob: s.dob ? s.dob.substring(0, 10) : "",
          gender: s.gender || "",
          phone: s.phone || "",
          email: s.email || "",
          address: s.address || "",
          fatherName: s.fatherName || "",
          motherName: s.motherName || "",
          parentPhone: s.parentPhone || "",
          // semester marks auto fill
          sem1: s.sem1 || "",
          sem2: s.sem2 || "",
          sem3: s.sem3 || "",
          sem4: s.sem4 || "",
          sem5: s.sem5 || "",
          sem6: s.sem6 || "",
        }));
        setPersistedMarks({
          sem3: s.sem3 || "",
          sem4: s.sem4 || "",
          sem5: s.sem5 || "",
          sem6: s.sem6 || "",
        });
```

```
        // Lock because data came from master database
        setIsProfileLocked(true);
        setHasProfile(true);
      }
      else {
        console.log("Enroll API empty. Fetching
/student/me...");
        const res = await
fetch("http://localhost:8000/student/me", {
          headers: {
            Authorization: `Bearer ${token}`,
          },
        });
        const data = await res.json();
        console.log("Profile API:", data);
        if (res.ok && data.success && data.data) {
          setIsProfileLocked(data.isProfileLocked ===
true);
          setHasProfile(true);
          setStatus(data.status || "");
          setRejectNote(data.rejectNote || "");
          setRejectSections(data.rejectSections || {});
          const profile = data.data;
          setForm({
            name: profile.name || "",
            enroll: profile.enroll || "",
            branch: profile.branch || "",
            year: profile.year || "",
            dob: profile.dob ? profile.dob.substring(0, 10)
: "",
            gender: profile.gender || "",
            phone: profile.phone || "",
```

```
            email: profile.email || "",
            address: profile.address || "",
            fatherName: profile.fatherName || "",
            motherName: profile.motherName || "",
            parentPhone: profile.parentPhone || "",
            sem1: profile.sem1 || "",
            sem2: profile.sem2 || "",
            sem3: profile.sem3 || "",
            sem4: profile.sem4 || "",
            sem5: profile.sem5 || "",
            sem6: profile.sem6 || "",
          });
          setPersistedMarks({
            sem3: profile.sem3 || "",
            sem4: profile.sem4 || "",
            sem5: profile.sem5 || "",
            sem6: profile.sem6 || "",
          });
          if (profile.photo) {
            setPhoto(`http://localhost:8000/uploads/${profi
le.photo}`);
          }
        }
      }
      fetchPermissions();
    } catch (err) {
      console.error("Error fetching profile:", err);
    } finally {
      setIsLoading(false);
    }
  };
```

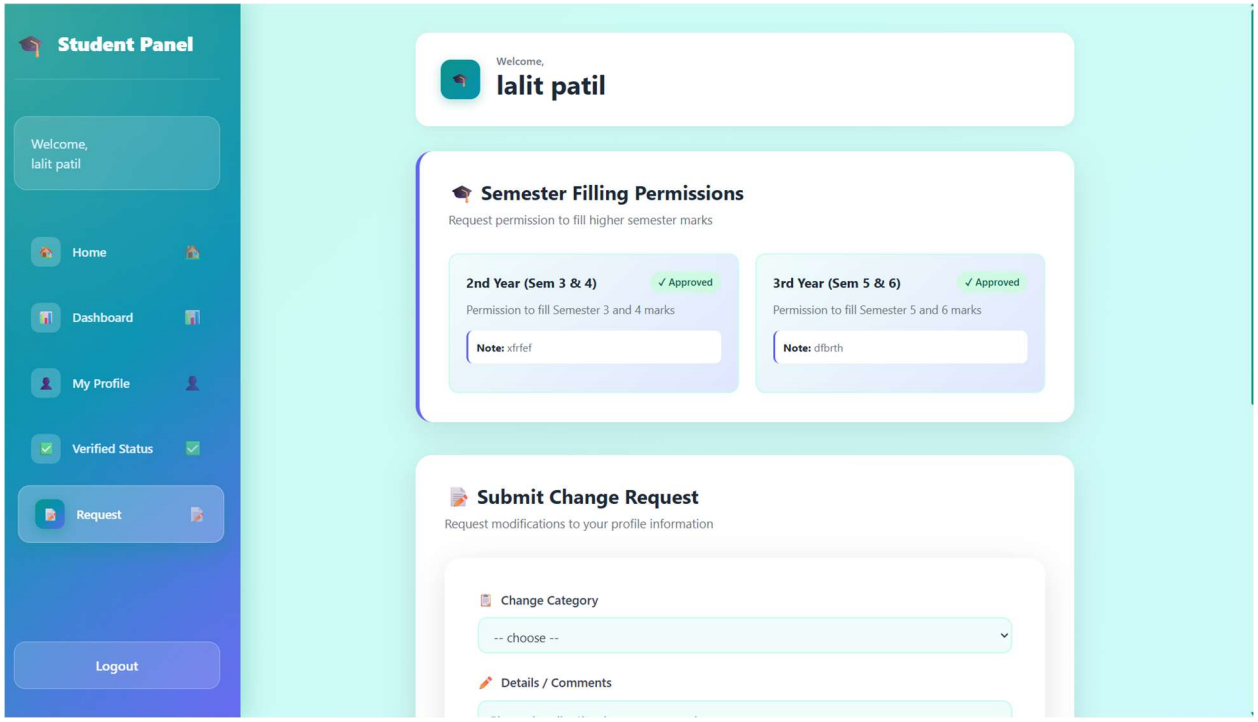


Fig 6.3: Student Request Interface

- Represents the Change Request Interface of the Blockchain Student Verification System
- Shows requests submitted by students for profile modifications
- Displays request status like:

1. Pending
2. Approved
3. Rejected

- Highlights requested changes with details and teacher feedback
- Helps students identify sections that need correction
- Allows submission of new change requests
- Categories include:

- Basic Details
- Contact Information
- Academic Details

- Provides easy navigation to profile and verification sections
- Helps students track and manage requests efficiently
- Ensures transparency and smooth communication between students and teachers

Code :

```
const fetchPermissions = async () => {
    try {
      const secondYearRes = await
fetch("http://localhost:8000/student/permission/status/2nd_
year", {
        headers: { Authorization: `Bearer ${token}` },
      });
      const secondYearData = await secondYearRes.json();
      if (secondYearData.success) {
        setSecondYearPermission(secondYearData.permission);
      }
      const thirdYearRes = await
fetch("http://localhost:8000/student/permission/status/3rd_
year", {
        headers: { Authorization: `Bearer ${token}` },
      });
      const thirdYearData = await thirdYearRes.json();
      if (thirdYearData.success) {
        setThirdYearPermission(thirdYearData.permission);
      }
    } catch (err) {
      console.error("Error fetching permissions:", err);
    }
  }
```

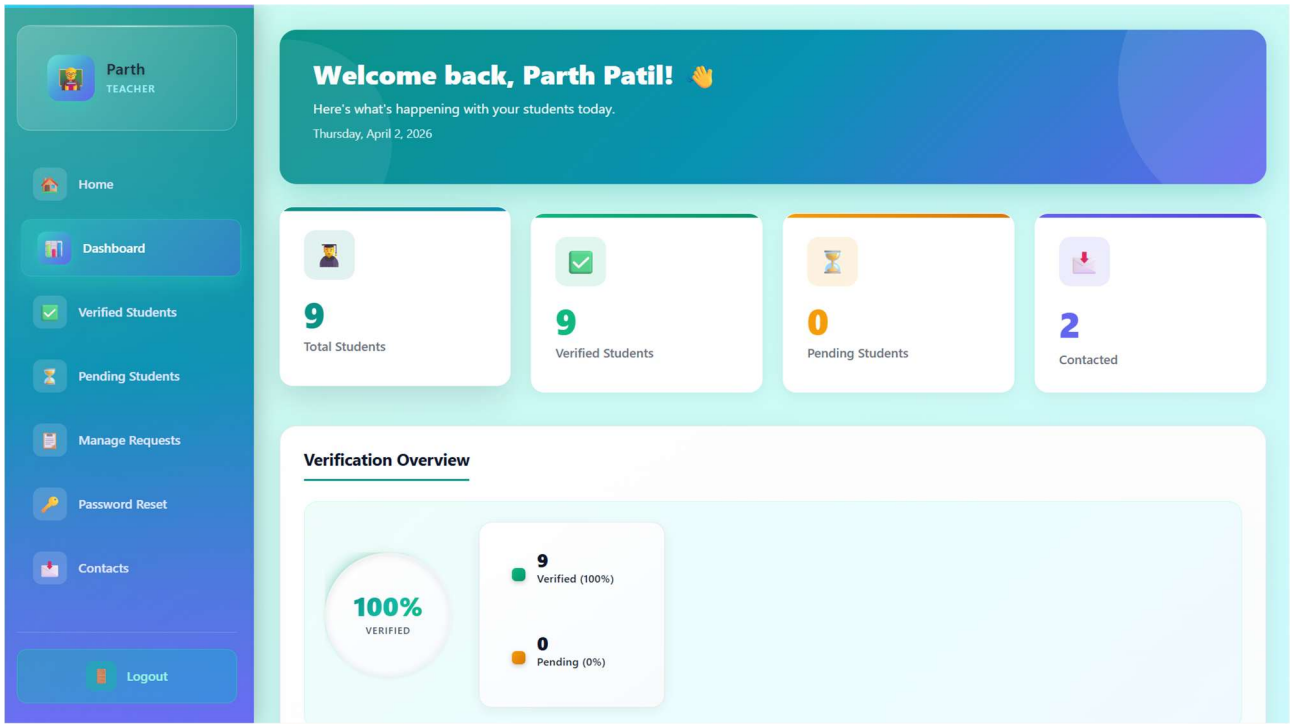


Fig 6.4: Teacher Dashboard Interface

- Represents the Teacher Dashboard Interface of Blockchain Student Verification System
- Provides overview of student profiles pending for verification
- Displays details like:

1. Student information
2. Submitted profiles
3. Verification status (pending/verified/rejected)

- Allows teachers to access and review student records section-wise
- Helps ensure accuracy and completeness of data
- Enables teachers to approve or reject profiles
- Allows giving feedback for corrections
- Helps manage student change requests
- Enables tracking of verification progress efficiently
- Includes navigation options for:

1. Pending Students

2. Verified Records

3. Requests

- Makes verification process easy and well-organized Overall, this interface helps teachers to perform verification tasks efficiently, reduces manual
- effort, and ensures accurate and reliable student record management.

Code: Dashboard Fetching

```
  useEffect(() => {
    const fetchStats = async () => {
      try {
        const res = await
fetch("http://localhost:8000/student/getst", {
          headers: {
            Authorization: `Bearer ${token}`,
          },
        });

        if (res.ok) {
          const students = await res.json();
          const verified = students.filter(s => s.verify
=== true).length;
          const pending = students.filter(s => s.verify ===
false).length
          setStats((prev) => ({
            ...prev,
            totalStudents: students.length,
            verifiedStudents: verified,
            pendingStudents: pending,
          }));
        }
        try {
```

```
          const contactRes = await
fetch("http://localhost:8000/contact/count", {
            headers: { Authorization: `Bearer ${token}` },
          });
          const contactData = await contactRes.json();
          if (contactRes.ok) {
            setStats((prev) => ({ ...prev, contacted:
contactData.count || 0 }));
          }
        } catch (err) {
          console.error("contact count error", err);
        }
      } catch (err) {
        console.error(err);
        dialog.error("Error", "Failed to fetch
statistics");
      } finally {
        setLoading(false);
      }
    };
    if (token) {
      fetchStats();
    }
  }, [token]);
```

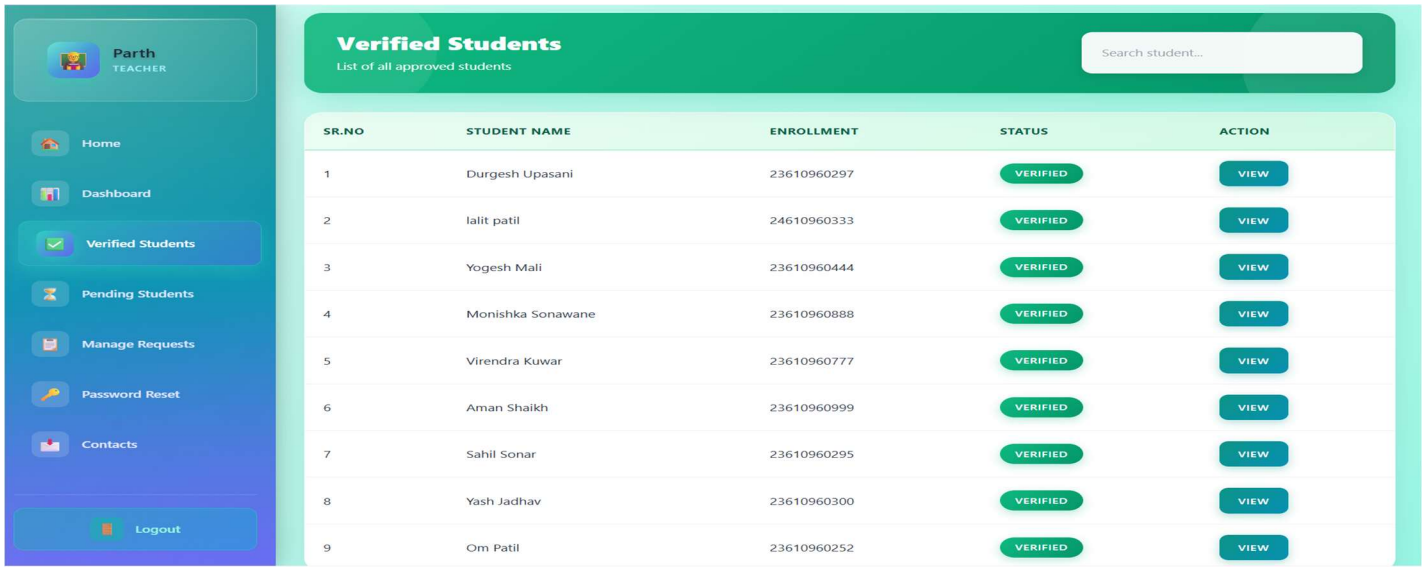


Fig 6.5: Verified student Interface

- Represents the Student Verified Interface in the Teacher Dashboard
- Shows all successfully verified student profiles
- Displays details like:

1. Student information
2. Verification status
3. Submitted profile data

- Allows teachers to view and access verified records easily
- Organizes data for easy search and filtering
- Verified data is final and cannot be modified (ensures integrity)
- Helps in quick review and tracking of student data
- Includes navigation options for:

1. Pending Students
2. Change Requests
3. Other dashboard features

- Maintains clear record of verified students
- Ensures transparency and efficient monitoring of the system

Code:

```
  useEffect(() => {
    const fetchPendingStudents = async () => {
      try {
        const res = await
fetch("http://localhost:8000/student/pending", {
          method: "GET",
          headers: {
            Authorization: `Bearer ${token}`,
          },
        });
        const data = await res.json();
        if (res.ok) {
          setStudents(data);
        } else {
          setError(data.message || "Failed to fetch
students");
        }
      } catch (err) {
        setError("Server error. Please try again later.");
        console.error(err);
      } finally {
        setLoading(false);
      }
    };
    fetchPendingStudents();
  }, [token]);
```

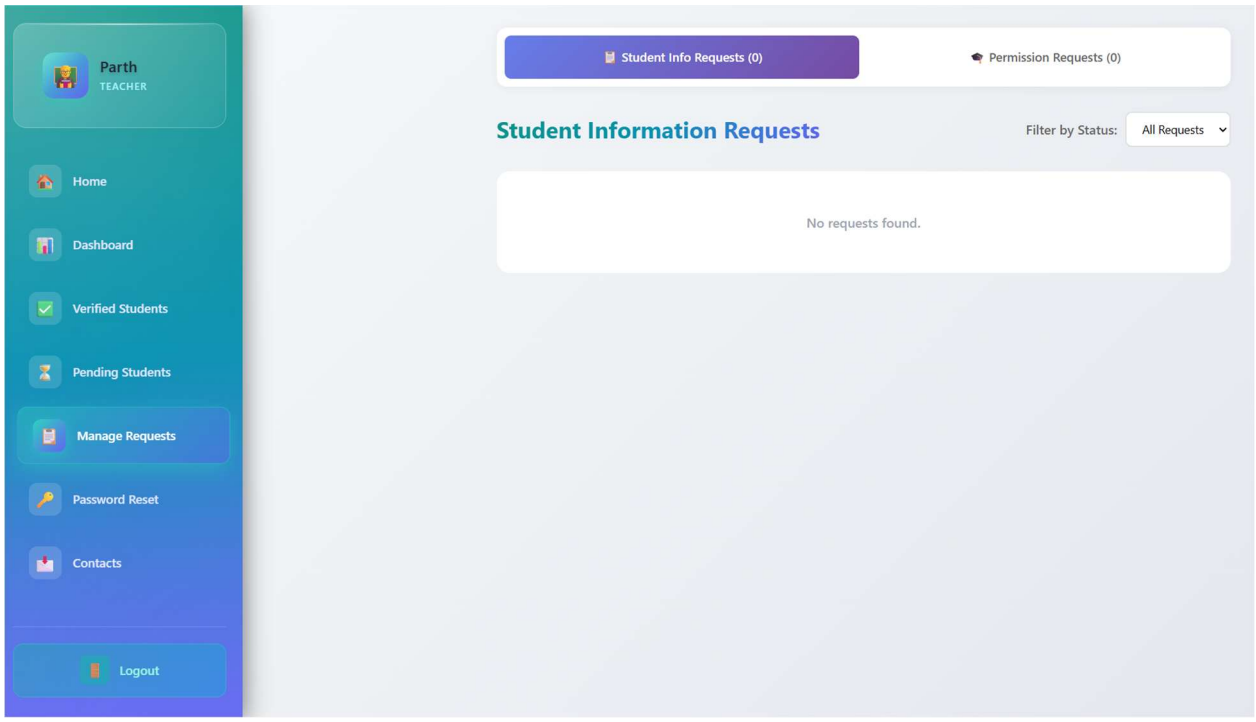
- 

Fig 6.6: Manage Request Interface

- Represents the Manage Request Interface in the Teacher Dashboard
- Shows all change requests submitted by students
- Displays details like:

1. Student information
2. Request category
3. Requested changes
4. Status (pending/approved/rejected)

- Allows teachers to review requests easily
- Enables approval or rejection of requests
- Teachers can provide feedback or notes for corrections
- Improves communication between students and teachers
- Includes navigation options for:

1. Pending Students
2. Verified Profiles

3. Other dashboard features

- Helps manage all requests efficiently
- Ensures transparency and accuracy of student data
- Provides a smooth and structured request handling process

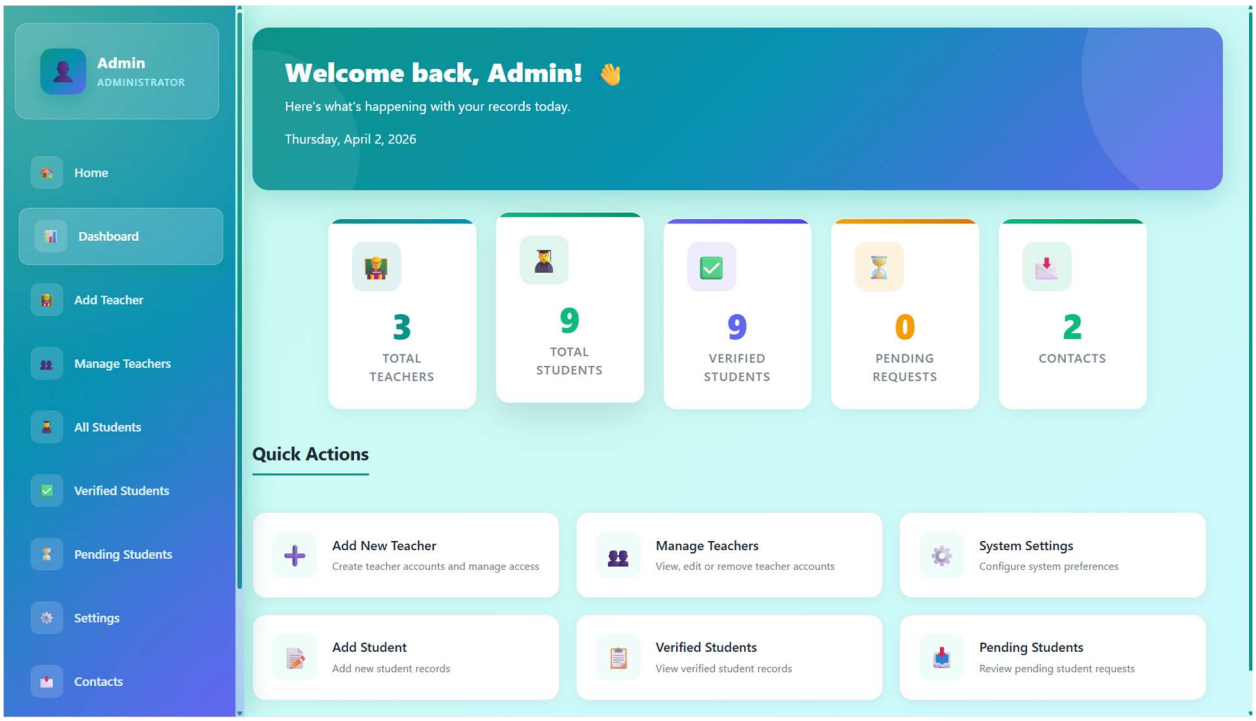


Fig 6.7: Admin Dashboard Interface

- Represents the Admin Dashboard Interface of the Blockchain Student Verification System
- Provides a complete overview of the system
- Displays details like:

1. Total number of students
2. Verified and pending profiles
3. System statistics

- Helps admin to monitor system performance and activities
- Left-side navigation includes:

1. Dashboard
2. Manage Students

3. Manage Teachers

4. Requests

5. Reports

6. Settings

- Allows admin to add, update, and delete records
- Enables monitoring of verification process
- Helps in handling system-related issues
- Provides reports and analytics for decision-making
- Ensures data security and system control
- Helps manage users and operations efficiently

Code:

```
  useEffect(() => {
    const fetchCounts = async () => {
            try {
        const studentRes = await
fetch("http://localhost:8000/student/getst", {
          headers: {
            Authorization: `Bearer ${token}`,
          },
        });
                if (studentRes.ok) {
          const students = await studentRes.json();
          const verified = students.filter(s => s.verify
=== true).length;
          const pending = students.filter(s => s.verify ===
false).length;
                    setStudentCount(students.length || 0);
          setVerifiedCount(verified);
          setPendingCount(pending);
        }
```

```
        const teacherRes = await
fetch("http://localhost:8000/teacher/teachers", {
          headers: {
            Authorization: `Bearer ${token}`,
          },
        });
        const teacherData = await teacherRes.json();
        if (teacherRes.ok) {
          setTeacherCount(teacherData.length || 0);
        }
        try {
          const contactRes = await
fetch("http://localhost:8000/contact/count", {
            headers: { Authorization: `Bearer ${token}` },
          });
          const contactData = await contactRes.json();
          if (contactRes.ok) {
            setContactCount(contactData.count || 0);
          }
        } catch (err) {
          console.error(err);
        }
      } catch (err) {
        console.error(err);
      } finally {
        setLoading(false);
      }
    };
    fetchCounts();
  }, [token]);
```

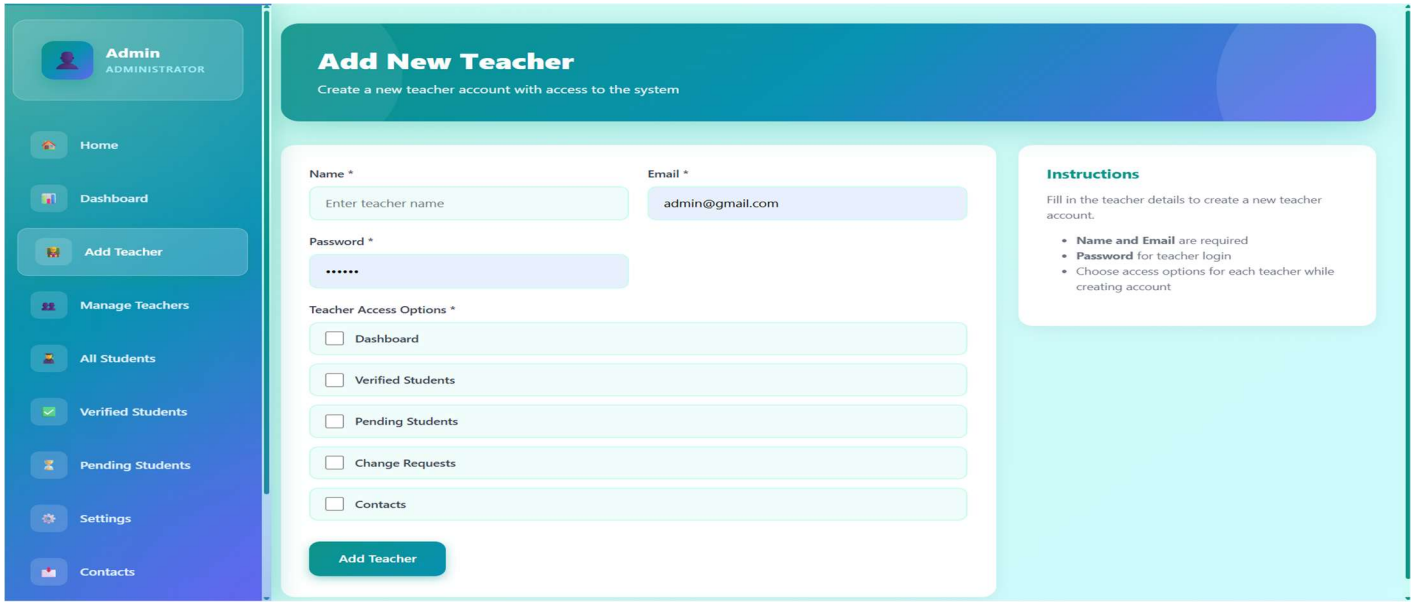


Fig 6.8: Admin add teacher Interface

- Represents the Admin Add Teacher Interface of Blockchain Student Verification System
- Allows admin to add new teacher accounts
- Includes input fields for:

1. Name
2. Email
3. Password

- Ensures all required details are filled correctly
- Designed to be simple and user-friendly
- Helps admin perform tasks quickly and easily
- Includes validation to avoid incorrect or incomplete data
- Stores teacher data securely in the database
- Enables teachers to log in and access their dashboard
- Helps in efficient management of teacher accounts
- Supports smooth functioning of the system

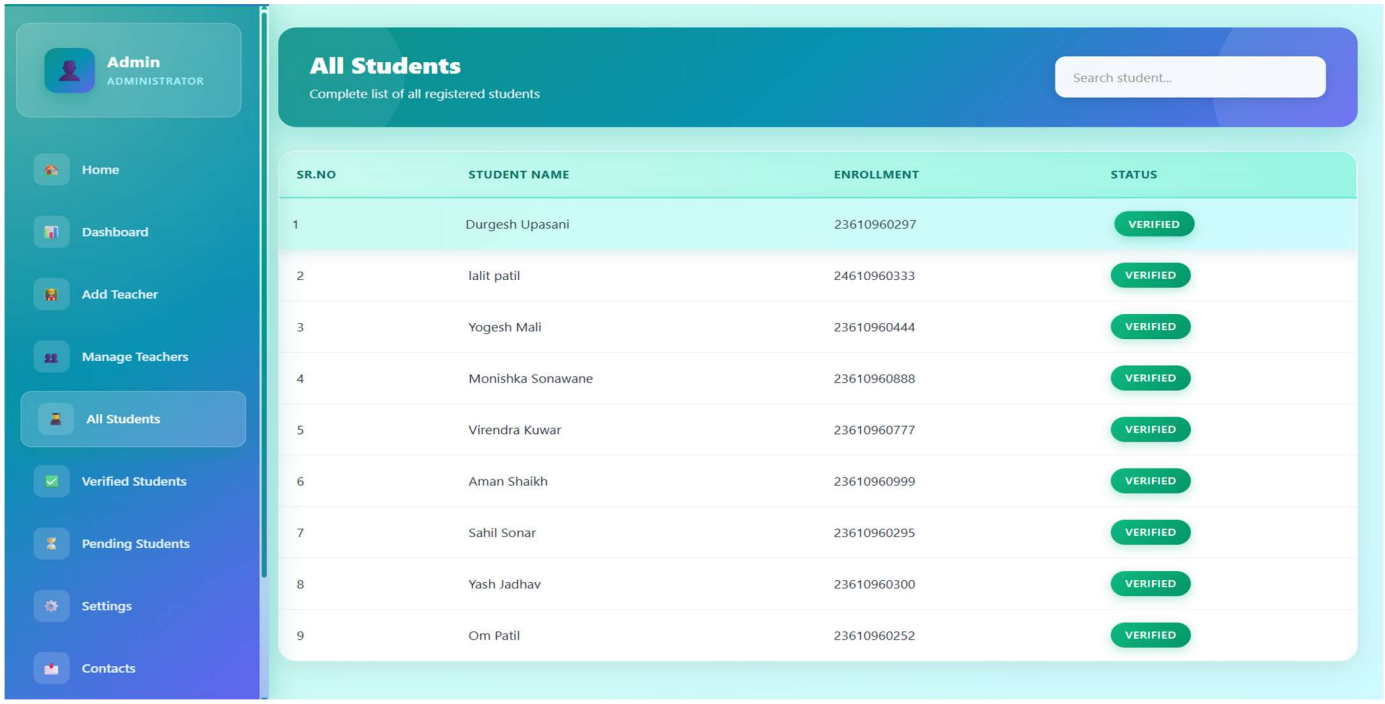


Fig 6.9: Admin all students Interface

- Represents the Admin All Students Interface of Blockchain Student Verification System
- Provides a complete overview of all registered students
- Displays details like:

1. Student name
2. Enrollment number
3. Profile status (verified/pending)
4. Other relevant information

- Allows admin to view and manage student records easily
- Includes search and filter options for quick access
- Enables admin to view detailed student information
- Designed to be simple and user-friendly
- Ensures smooth navigation and efficient data management
- Helps in monitoring student data and verification status
- Maintains accurate and organized records
- Ensures proper control over system data

Code:

```
const fetchAllStudents = async () => {
  try {
    const res = await fetch("http://localhost:8000/student/getst", {
      headers: { Authorization: `Bearer ${token}` },
    });
    const data = await res.json();
    if (res.ok) {
      setStudents(data);
    }
  } catch (err) {
    console.log(err);
  } finally {
    setLoading(false);
  }
};
useEffect(() => {
  fetchAllStudents();
}, []);
```

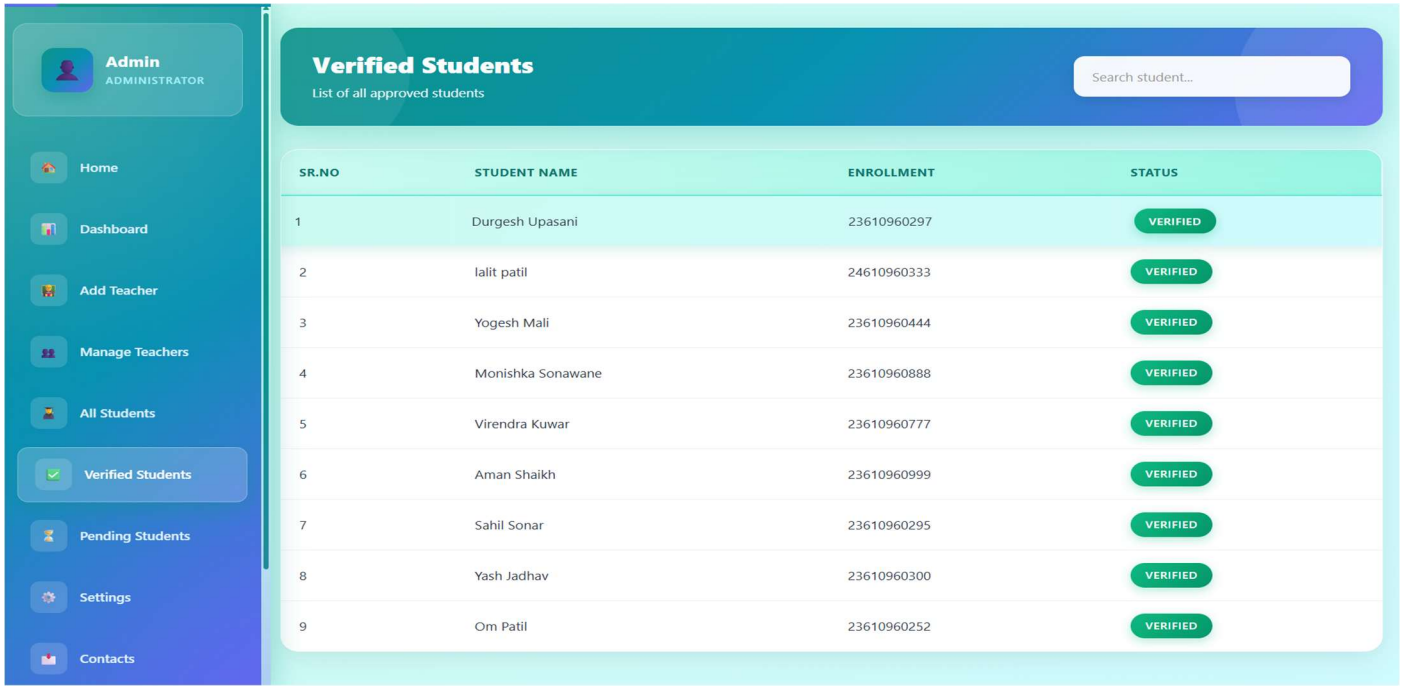


Fig 6.10: Admin verified students Interface

- Represents the Admin Verified Students Interface of the Blockchain Student Verification System
- Shows list of successfully verified student profiles
- Displays details like:

1. Student name
2. Enrollment number
3. Verification status

- Allows admin to view verified records easily
- Verified data is final and cannot be modified
- Ensures data integrity and security
- Helps admin to monitor and maintain records
- Includes search and filter options for quick access
- Designed to be simple and user-friendly
- Enables efficient management of verified data
- Ensures transparency and data authenticity

CHAPTER 7

TESTING

7.1 Test Plan: -

Section	Details
Test_Plan_ID	TP BSYS-001
Objective: -	The objective of this test plan is to verify the Blockchain student Verification system work-correctly, securely and efficiently. It ensures that all modules such as student profile management, teacher verification and admin controls Functions as expected errors.
Scope: - **(In Scope)**	• Student Registration & Login. • Profile Form submission. • Blockchain Encryption & Storage. • Teacher Verification System • Change Request Module. • Admin Dashboard. • Password Reset System.
out of scope: -	• External third-party integrations. • Hardware-related testing.
Test steps:	1. Student logs into the system. 2. Student fills & submits profile. 3. Teacher review & verifies / rejects profile. 4. Teacher store data in blockchain database
Test Approach:	• Functional testing for alt 3 module. • Positive & negative testing for validation • Integration testing for API. • UI/UX testing for user friendly interface
Entry criteria:	• All projects' modules are developed. • Test environment is ready. • Test data is available.
Exit criteria: -	• All test cases executed successfully.

	• System meets expected performance. • Test reports are completed.
Test Environment: -	• OS: Windows. • Browser: - Google chrome, firefox. • Database: - MangoDB. • Tools: -vscode, postMon • Frontend: -React • Backend: -Node.js
Role & Responsibilities	1. Role: -Tester. Responsibilities: - Executes test cases & report bugs. 2. Role: - Developer. Responsibilities: - Fix issues & update code. 3. Role: - Project Guide Responsibilities: - Approve final test summary
Risk: -	• System crash. • Data loss of corruption. • Time constraints.

1. **Test Cases for Student Login Module: -**

Test Case ID	Objective	Steps	Expected Output	Actual Output	Result
TC01	Login with valid credentials	Enter valid enroll no & password	Login successful	Login successful	Pass
TC02	Login with invalid credentials	Enter wrong details	Error message	Error shown	Pass
TC03	JWT token generation	Login and check token	Token generated	Token generated	Pass
TC04	Access protected route	Use valid token	Access granted	Access granted	Pass
TC05	Logout functionality	Click logout	User logged out	Logout successful	Pass
TC06	Check empty login fields	Leave enroll & password blank and click login	Validation error shown	Error displayed	Pass
TC07	Check invalid email format	Enter wrong Enroll no format	Error message for invalid Enroll	Error shown	Pass
TC08	Password masking	Enter password in field	Password should be hidden (***)	Password hidden	Pass
TC09	Session timeout	Stay inactive after login	User logged out automatically	Session expired	Pass
TC10	Unauthorized access	Access dashboard without login	Access denied / redirect to login	Access denied / redirect to login Redirected	Pass

Test Summary Report Identifier:

TSR_LOGIN_01 – Unique ID for the test summary report

1. Summary:

Testing of the Authentication Module was conducted to verify secure login functionality, JWT-based authentication, session handling, input validation, and access control. Multiple test cases were executed including valid login, invalid credentials, empty fields, token generation, password masking, session timeout, unauthorized access, and logout functionality.

2. Variances:

All test cases were executed as planned. No major deviations were observed during testing. Minor variations were noticed in session timeout depending on system inactivity duration, but they did not affect overall functionality.

3. Comprehensiveness:

Testing covered all major scenarios including both positive and negative cases. Validation checks, authentication flow, security features, and access restrictions were thoroughly tested to ensure system reliability.

4. Results:

10 Passed: Valid login, invalid login, JWT token generation, protected route access, logout, empty field validation, invalid enroll format, password masking, session timeout, unauthorized access

5. Evaluation:

The Authentication Module is functioning correctly and securely. All functionalities are working as expected, and no critical defects were identified. The module is stable and ready for integration with other system modules.

6. Activities:

Requirement analysis, test case design, environment setup, execution of authentication test cases, validation checks, JWT verification, session handling testing, and access control verification.

7. Approvals:

Approved by Test Lead, Project Guide, and Development Team for further integration and system testing.

Defect Report: -

Field	Details
Defect ID	DEF-LOGIN-002
Module	Login Module
Test Case ID	TC-LOGIN-008
Defect Title	Special characters allowed in username field
Description	The system allows login attempt with special characters in username instead of restricting or validating input properly.
Steps to Reproduce	1. Open login page 2. Enter username with special characters (e.g., shree@#) 3. Enter valid password 4. Click login
Expected Result	System should restrict special characters or show validation error message
Actual Result	System accepts input and proceeds without proper validation
Severity	Medium
Priority	Medium
Status	Open
Reported By	Test Engineer
Assigned To	Developer Team

2. **Test Cases for Student Module: -**

Test Case ID	Objective	Steps	Expected Output	Actual Output	Result
TC01	Fill profile form	Enter valid data	Data accepted	Data accepted	Pass
TC02	Submit profile	Click submit	Profile saved	Profile saved	Pass
TC03	Upload photo	Upload image	Image uploaded	Uploaded	Pass
TC04	Empty field validation	Leave fields blank	Error message	Error shown	Pass
TC05	View profile	Open profile page	Data displayed	Data displayed	Pass

3. **Test Cases for Teacher Module: -**

Test Case ID	Objective	Steps	Expected Output	Actual Output	Result
TC06	View pending students	Open dashboard	List displayed	List shown	Pass
TC07	Verify student	Click verify	Status updated	Updated	Pass
TC08	Reject student	Click reject	Feedback sent	Feedback sent	Pass
TC09	Approve change request	Review request	Approved	Approved	Pass
TC10	View verified list	Open verified tab	List shown	List displayed	Pass

TEST SUMMARY REPORT

Report Identifier:

TSR_ADMIN_02 – Unique ID for Admin Module Test Summary

1. Summary:

Testing of the Admin Module was conducted to verify functionalities such as viewing pending students, verifying student records, rejecting students with feedback, approving change requests, and viewing verified student lists. The goal was to ensure smooth workflow and accurate status updates within the system.

2. Variances:

All test cases were executed successfully without any major deviations. Minor differences in UI response time were observed but did not impact functionality.

3. Comprehensiveness:

Testing covered all major admin operations including both data viewing and decision-based actions (verify, reject, approve). All workflows were validated to ensure proper system behavior.

4. Results:5 Passed

- View pending students
- Verify student
- Reject student
- Approve change request
- View verified list

5. Evaluation:

The Admin Module is functioning correctly. All features are working as expected, and the system properly updates and displays student status. No critical issues were found.

6. Activities:

Requirement analysis, test case execution, verification of student status updates, validation of approval/rejection workflows, and UI testing of dashboard features.

7. Approvals:

Approved by Test Lead, Project Guide, and Development Team for further system integration.

DEFECT REPORT

Field	Details
Defect ID	DEF-TEACHER-02
Module	Teacher Module
Test Case ID	TC-Teacher-008
Defect Title	Delay in updating student verification status
Description	The system shows a delay in updating the student verification status after the admin clicks the verify button. The status does not refresh immediately on the dashboard.
Steps to Reproduce	1. Open Teacher Dashboard 2. Go to Pending Students 3. Click on Verify button for any student 4. Check the status immediately
Expected Result	Student status should update instantly to “Verified” and reflect on the dashboard
Actual Result	Status update is displayed and requires manual refresh to display updated result
Severity	Medium
Priority	Medium
Status	Open
Reported By	Test Engineer
Assigned To	Developer Team

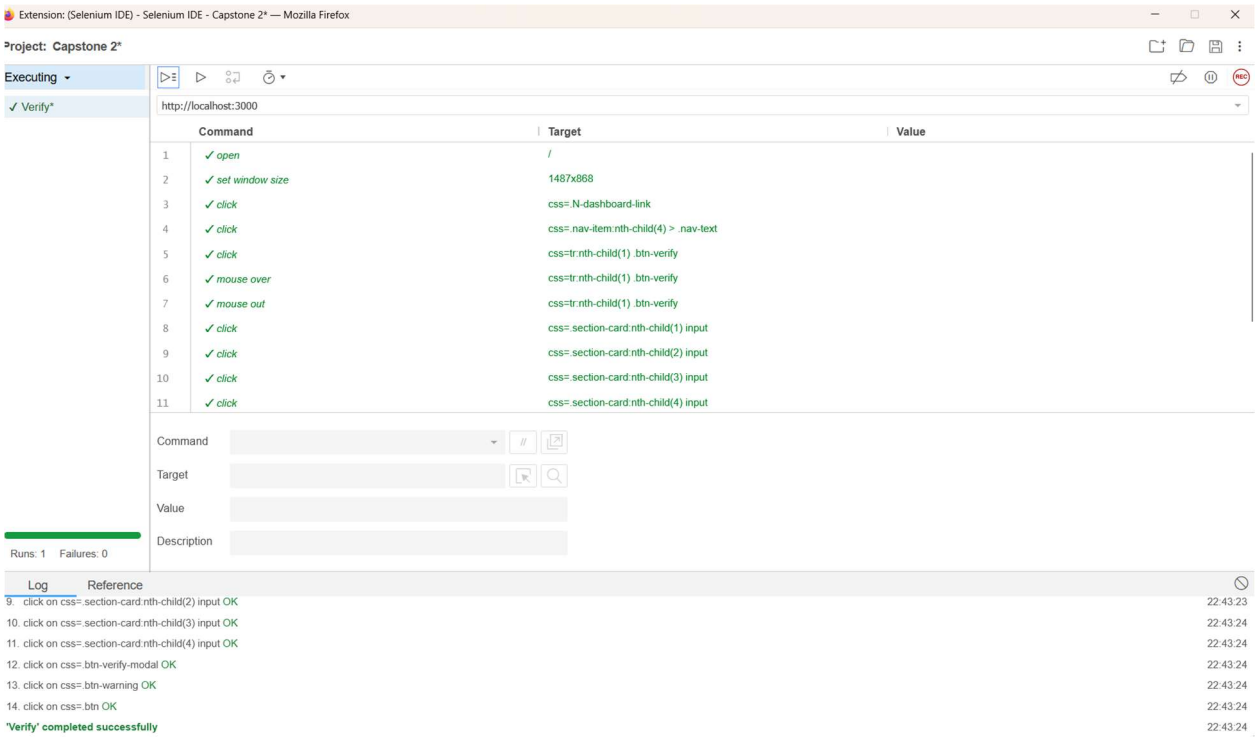
4. **Test Cases for Admin Module: -**

Test Case ID	Objective	Steps	Expected Output	Actual Output	Result
TC11	View all students	Open dashboard	Data displayed	Data shown	Pass
TC12	Add teacher	Enter details	Teacher added	Added	Pass
TC13	Edit teacher	Update info	Data updated	Updated	Pass
TC14	Delete teacher	Remove teacher	Deleted	Deleted	Pass
TC15	View analytics	Open dashboard	Stats displayed	Stats shown	Pass

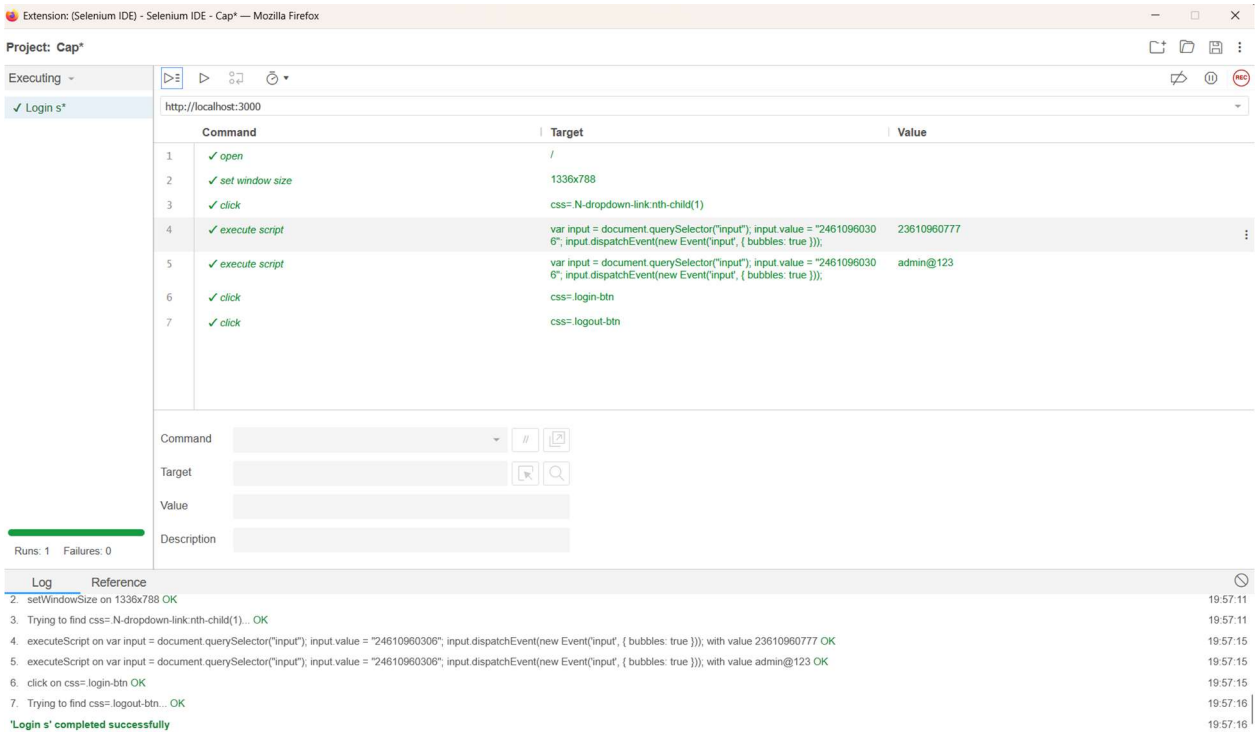
5. **Test Cases for Password reset Module: -**

Test Case ID	Objective	Steps	Expected Output	Actual Output	Result
TC16	Request reset	Enter Enroll No	Request sent to teacher dashboard	Teacher approve request	Pass
TC17	Reset password	Enter new password	Password updated	Updated	Pass
TC18	Invalid password	Enter wrong token	Error message	Error shown	Pass
TC19	Use old password	Enter Old password	Access denied	Denied	Pass
TC20	Login after reset	Login with new password	Login successful	Success	Pass
Test Case ID	**Objective**	**Steps**	**Expected Output**	**Actual Output**	**Result**
TC21	Submit request	Fill request form	Request saved	Saved	Pass

TC22	View request status	Open request page	Status shown	Displayed	Pass
TC23	Approve request	Teacher approves	Changes allowed	Allowed	Pass
TC24	Reject request	Teacher rejects	Feedback shown	Shown	Pass
TC25	Track history	Open history	History displayed	Displayed	Pass



7.1 Selenium Verify



7.2 Selenium Student Login

CHAPTER 8
CONCLUSION AND FUTURE SCOPE

Conclusion: -

In conclusion, the Blockchain for Student record System provides a secure, efficient, and user-friendly platform for managing and verifying student records in educational institutions. The system enables students, teachers, and administrators to interact in a structured manner and perform their respective tasks effectively.

By implementing blockchain-inspired technology and cryptographic encryption, the system ensures data integrity, transparency, and security. Once the student data is verified, it becomes immutable, preventing any unauthorized modification or tampering. This increases trust and reliability in academic records.

The system reduces manual work, minimizes human errors, and simplifies the verification process. Students can easily manage their profiles and track verification status, while teachers can efficiently review and verify records. Administrators can monitor system activities and manage users effectively.

Overall, the system improves educational data management, enhances security, and provides a reliable solution for maintaining and verifying student records in a modern digital environment.

Future Scope: -

1. The system can be expanded for use across multiple colleges and universities at a national level
2. Integration with real blockchain platforms (such as Ethereum) for enhanced decentralization
3. Multi-language support can be added for accessibility in different regions
4. Mobile application (Android/iOS) can be developed for better usability
5. SMS and email notification system can be integrated for verification updates
6. Cloud-based deployment can be implemented for better scalability and accessibility
7. Advanced security features like Two-Factor Authentication (2FA) can be added
8. AI-based analytics can be introduced for performance tracking and insights

CHAPTER 9

REFERENCES AND BIBLIOGRAPHY

References: -

• Teachers and academic staff with whom we discussed requirements and gained knowledge about student record management and verification processes:

1. Faculty Members of College.
2. Project Guide / Mentor.
3. Academic Administrative Staff.

• Students (users) who provided information about existing record systems and challenges: Students from various departments and academic years.

Bibliography: -

• Online Student Management and Verification Systems – Various educational websites and portals

• Research Papers on Blockchain Technology and Data Security

• MERN Stack Development Resources (MongoDB, Express.js, React, Node.js)

• JSON Web Token (JWT) Authentication Documentation

• SHA-256 Encryption and Cryptography Resources

• HTML, CSS, JavaScript Documentation (W3Schools, MDN)

• Official Documentation of Node.js, React, and MongoDB

• Articles and blogs related to Blockchain-based applications

• GitHub repositories and open-source projects on student management systems.